Gemini CLI – Enhancement Proposal: Tool Execution Rollback and Undo System

April 2026

Authors:

Iram (18ih2@queensu.ca), Lena (22bsn3@queensu.ca), Maciek (20mjb21@queensu.ca), Mack (22NWD3@queensu.ca), Martin (martin.allerdissen@queensu.ca)

Abstract

This report proposes a Tool Execution Rollback and Undo System for Gemini CLI, Google's open-source command-line interface for the Gemini family of large language models. The enhancement addresses a gap in the system's undo ergonomics and scheduler-integrated rollback. Robust pre-execution safeguards exist through the PolicyEngine and confirmation workflow, but the default experience does not provide stack-based or turn-based /undo semantics. Optional checkpointing with /restore [7] can revert project files when enabled, but it differs from the proposed Rollback Manager in prerequisites, scope, and workflow. We present the enhancement through two non-trivial use cases covering single-file undo and multi-file turn rollback. We evaluate its effects on maintainability, evolvability, testability, and performance, and describe its integration into the high-level architecture and the low-level six-layer architecture of packages/core. Two implementation alternatives are analyzed using the SEI SAAM methodology: a Snapshot-Based Rollback system and an Operation-Log Based Rollback system. The SAAM analysis identifies four stakeholder groups and six non-functional requirements. It finds that the Snapshot-Based approach is the stronger overall choice due to superior reliability, lower maintenance burden, and automatic compatibility with the existing tool ecosystem including MCP-based tools.

Introduction and Overview

Gemini CLI is an open-source, terminal-based agentic coding assistant developed by Google [3, 4]. As established in our prior analyses [1, 2], the system is organized as a monorepo with two primary packages: packages/cli (the terminal-facing interface) and packages/core (the backend orchestration engine). The core subsystem coordinates all communication with the Gemini API, manages tool execution through a formal state machine called CoreToolScheduler, enforces safety policies through the PolicyEngine and MessageBus, and supports multi-agent delegation through the AgentRegistry and LocalAgentExecutor [2].

A central feature of Gemini CLI is its ability to autonomously execute tools that modify the local development environment. The built-in tool set includes file operations, shell command execution, web fetching, and directory exploration. External tools can be added through the Model Context Protocol (MCP) [10]. All potentially destructive operations are gated by the PolicyEngine and require user approval through the confirmation workflow mediated by the MessageBus [2]. This approval mechanism provides a pre-execution safety net.

Post-execution recovery is limited in typical sessions where checkpointing is disabled by default. Approved tool effects are not automatically reversible through a dedicated undo stack. When checkpointing is enabled [7], the CLI can record a shadow Git snapshot of project files and support /restore to revert files and conversation, but that path is optional and differs in user experience from the proposed /undo and /undo-turn behavior. Where neither checkpointing nor other recovery applies,

changes remain as written unless recovered manually. In agentic workflows where the model chains multiple tool calls within a single turn, the developer must manually identify and restore every modified file when changes are incorrect, undermining the productivity gains the tool is designed to provide.

This report proposes a Tool Execution Rollback and Undo System that addresses this gap. Section 2 motivates the enhancement. Section 3 presents two use cases with sequence diagrams. Section 4 analyzes effects on system quality attributes. Section 5 describes architectural changes. Section 6 introduces two implementation alternatives. Section 7 presents the SAAM analysis. Section 8 identifies impacted files. Section 9 discusses feature interactions and testing plans. Section 10 addresses risks. Section 11 concludes with a recommendation.

Enhancement Motivation and Selection Process

The proposed enhancement is motivated by an asymmetry in Gemini CLI's current safety model. Pre-execution safeguards are robust through the PolicyEngine and confirmation workflow [9], but in the default configuration with checkpointing off, approved changes are not automatically reversible through a dedicated undo stack. The practical effect is a one-way commitment, and the cost of a mistaken approval scales with the number and complexity of tool calls in a session.

Existing recovery options do not close this gap for typical configurations. The /restore command can revert files when checkpointing is explicitly enabled [7], and some flows can revert edits through conversation history, but neither provides default-on, scheduler-integrated /undo and /undo-turn semantics. Git provides a manual escape hatch when projects use version control, but requiring users to understand git stash or git checkout to recover from an AI assistant's mistake defeats the tool's ease of use. The multi-agent framework [2] amplifies the risk further, since subagent tool calls may see reduced or non-interactive confirmation depending on policy and execution path.

Process for Identifying the Enhancement

Our group identified this enhancement through a systematic gap analysis of the safety architecture documented in A1 and A2. In A1, we described the approval gate as the primary safety mechanism. In A2, we decomposed this into three subsystems: PolicyEngine, MessageBus, and HookSystem, and documented the CoreToolScheduler state machine. This analysis revealed that all safety checks occur before the executing state transition. No mechanism exists after execution completes. We then surveyed the Gemini CLI issue tracker on GitHub [4] and found multiple user-reported incidents where approved tool actions produced unintended results, confirming the practical significance of this gap. After evaluating several candidate enhancements including parallel tool execution and pluggable LLM backends, we selected the rollback system for three reasons: it addresses a documented user need, it touches multiple architectural layers identified in our prior analyses, and it is not provided today as a default-on, scheduler-integrated undo subsystem with a dedicated session rollback store.

Use Cases

The following use cases assume the Snapshot-Based Rollback approach from Alternative A for concreteness.

Use Case 1: Undoing an Incorrect Single-File Refactoring

A developer asks Gemini CLI to refactor a function in their codebase. The agent reads the file, proposes changes involving optional chaining and restructured error handling, and the user approves the edit. After the modification is applied, the developer runs the test suite and discovers the refactoring

has broken several tests due to an incorrect assumption about null propagation. The developer issues the /undo command to revert the last tool action. The Rollback Manager retrieves the pre-edit snapshot, restores the file, and confirms the restoration. The complete interaction is shown in Figure 1.

Actors: User, CLI Frontend, GeminiClient, CoreToolScheduler, Rollback Manager, Host OS/Filesystem, ToolExecutor, Built-in Tools.

Preconditions: The user has an active session with rollback enabled. At least one file-modifying tool action such as write_file or replace has successfully completed. Prior to that tool's execution, a corresponding RollbackPoint was captured via the CoreToolScheduler and indexed by the Rollback Manager.

Flow: (1) User types "/undo" in the terminal. (2) CLI Frontend sends an undoLastAction() request to GeminiClient. (3) GeminiClient invokes RollbackManager.undoLast(). (4) Rollback Manager retrieves the most recent RollbackPoint from its in-memory index, which contains the originalContent, filePath, and toolCallId. (5) Rollback Manager writes the originalContent directly to the Host OS/Filesystem. This bypasses the ToolExecutor pipeline to guarantee exact byte-level restoration. (6) Rollback Manager marks the RollbackPoint as consumed and emits a RollbackCompleted event via coreEvents. (7) GeminiClient receives the event and appends a system note to the GeminiChat history so the model's context stays consistent with the restored filesystem state. (8) CLI Frontend renders a rollback-summary Ink component showing the restored file path and a diff of the reversal.

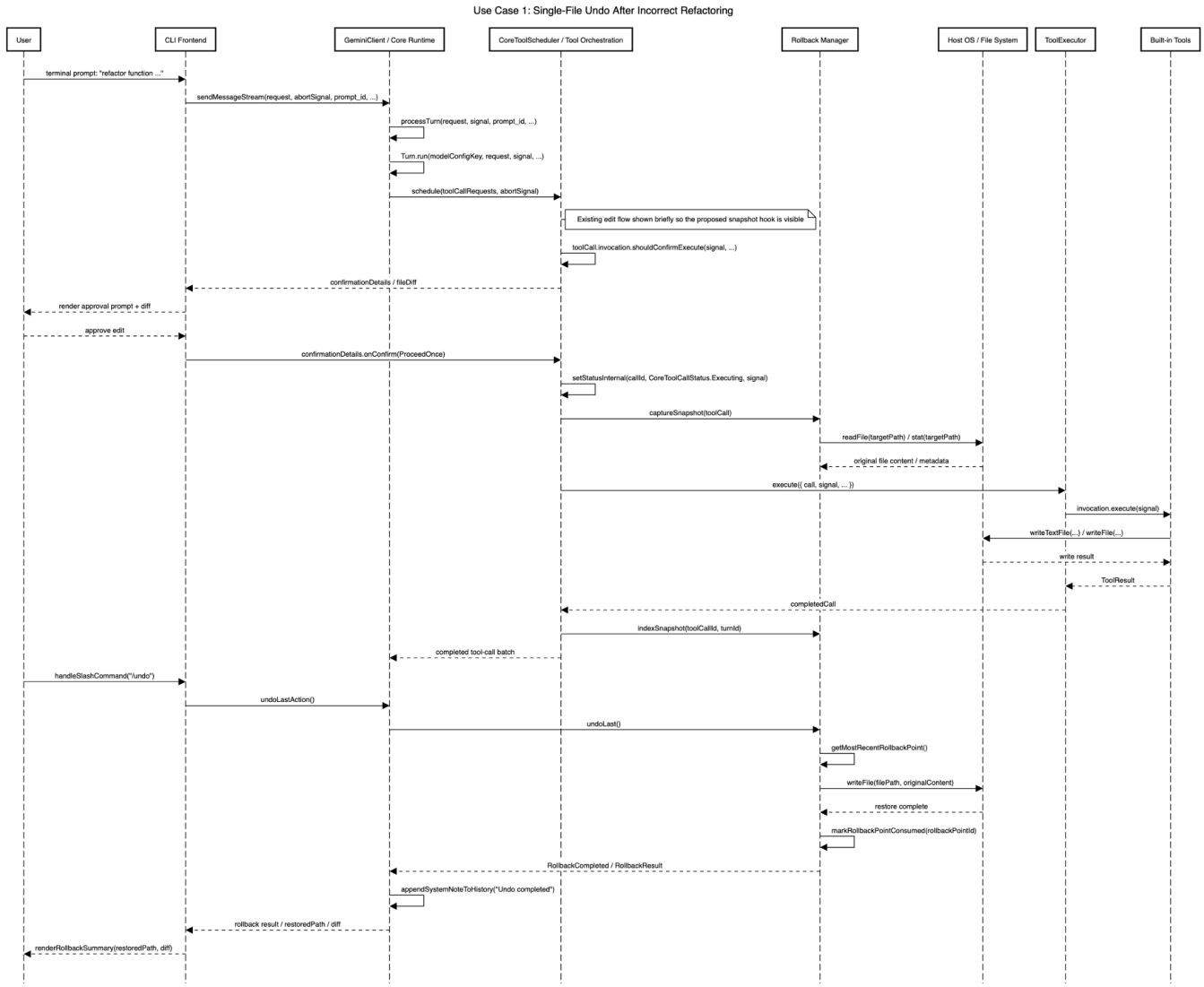

Figure 1: *Sequence diagram for Use Case 1. As shown in this diagram, rollback operations are executed in the same centralized orchestration flow as observed in the concrete architecture with control being executed by the Core Runtime and the execution being delegated by the Rollback Manager.*

Postconditions: The file is restored to its pre-edit state byte-for-byte. The RollbackPoint is marked consumed and cannot be undone again. The conversation history retains the full record of the original edit and the undo action for auditability.

Use Case 2: Rolling Back a Multi-File Project Restructuring

A developer asks Gemini CLI to reorganize their project by moving utility functions into a shared module, updating import paths across multiple files, and creating a new barrel export file. The agent executes a sequence of tool calls: creating directories via write_file, moving three files using write_file plus deleting the originals, updating import statements in five dependent files with replace, and creating an index.ts barrel file via write_file. After reviewing the results, the developer decides the approach is wrong and issues /undo-turn to roll back all tool actions from the current turn. The complete interaction is shown in Figure 2.

Actors: User, CLI Frontend, GeminiClient, CoreToolScheduler, Rollback Manager, Host OS/Filesystem, ToolExecutor, Built-in Tools, PolicyEngine.

Preconditions: The user has an active session with rollback enabled. The CoreToolScheduler has completed a sequence of file-modifying tool calls such as write_file, replace, and delete in a single turn. Each tool call triggered a captureSnapshot and indexSnapshot call to the Rollback Manager prior to its execution.

Flow: (1) User types "/undo-turn" in the terminal. (2) CLI Frontend sends undoTurn(currentTurnId) to GeminiClient. (3) GeminiClient delegates the call to RollbackManager.undoTurn(currentTurnId). (4) Rollback Manager retrieves all relevant rollback data using getRollbackPointsForTurn(currentTurnId) and calculates the reverse execution sequence via computeReverseExecutionOrder(points). (5) Rollback Manager consults the PolicyEngine using checkRollbackPolicy("rollback_operations", currentTurnId). If an enterprise override requires confirmation, the CLI Frontend prompts the user before processing continues. (6) Rollback Manager enters a loop to process each RollbackPoint in reverse order. It restores modified files via writeFile, removes created files using unlink, recreates files that were deleted during the turn via writeFile, and removes any directories that were created during the turn and are now empty using rmdir. (7) Rollback Manager emits TurnRollbackCompleted / RollbackResult with a summary of all reverted operations. (8) GeminiClient updates the conversation context using appendSystemNoteToHistory("Turn rollback completed"). (9) CLI Frontend receives the completion signal and renders a rollback summary showing all reverted operations and restored files.

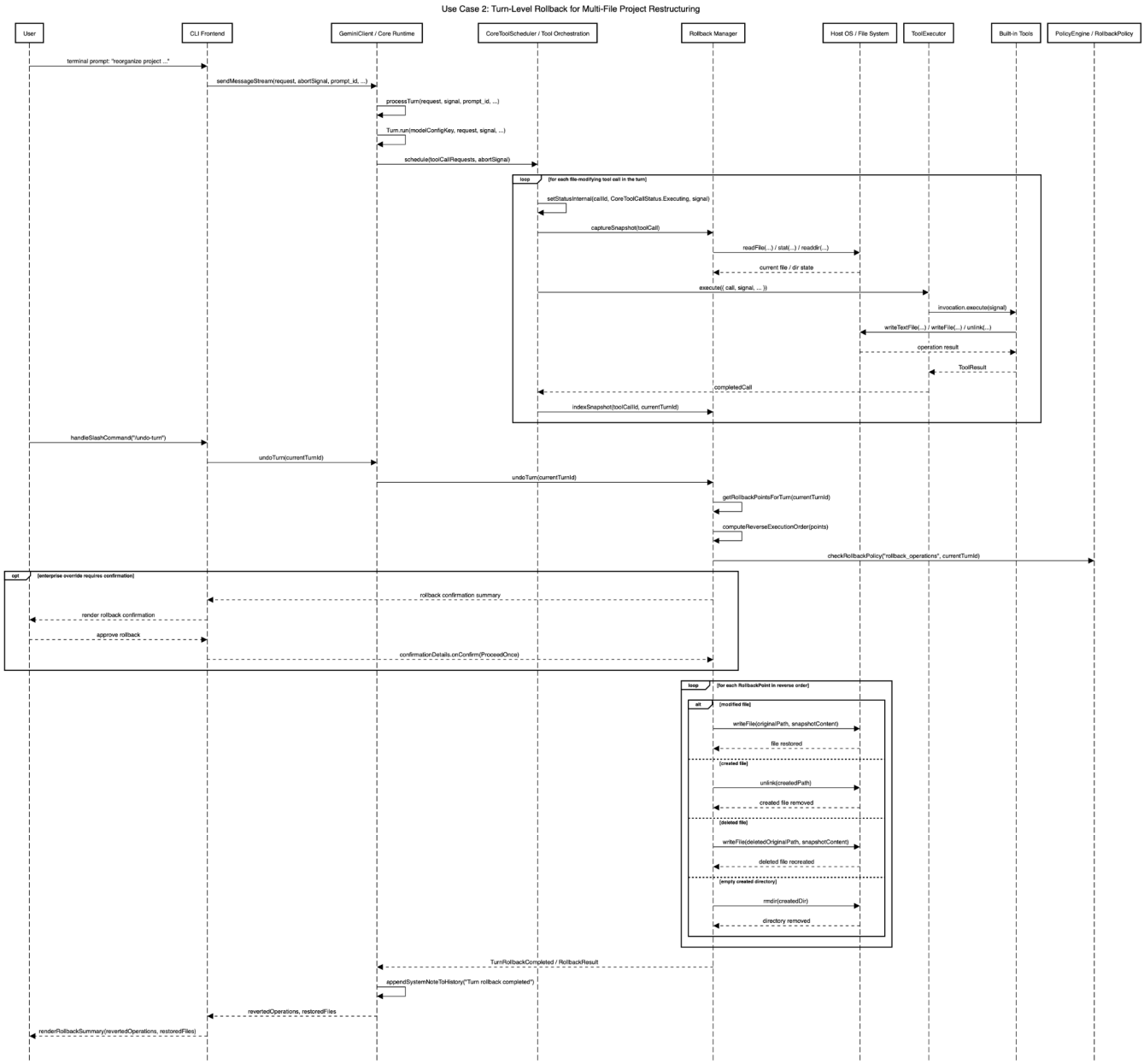

Figure 2: *Sequence diagram for Use Case 2. The upper portion shows the snapshot capture loop during the original turn. The middle portion shows the optional enterprise approval path. The lower portion shows the reverse-order restoration loop with branches for modified files, created files, deleted files, and empty directories.*

Postconditions: All file-based changes from the turn are restored to their exact pre-turn state. Non-file side effects such as external commands or network operations are not reversed. The conversation history is updated to reflect the rollback so the model's context stays synchronized with the local environment.

Effects on System Quality Attributes

This section analyzes how the proposed enhancement affects four key quality attributes of the Gemini CLI architecture, as described by Garlan and Shaw [6] and evaluated in the SAAM framework [5].

Maintainability

The Snapshot-Based Rollback Manager is designed as a self-contained subsystem within packages/core in a new src/rollback/ directory, with a narrow interface to existing components. Bugs in the rollback subsystem can be isolated and fixed without modifying the tool execution pipeline or safety subsystem. The integration with tool execution is limited to a single hook point: snapshot capture immediately before ToolExecutor.execute(), which ensures both CoreToolScheduler and the event-driven Scheduler share the same behavior. The enhancement does not require changes to any existing tool implementation, since built-in tools and MCP tools participate in rollback automatically through external snapshot capture. The fail-open design ensures that rollback subsystem failures cannot cascade into the critical tool execution path.

Evolvability

The enhancement introduces stable extension points. The SnapshotStore abstraction can be extended to support alternative storage backends such as Git-based snapshots or cloud-backed storage without modifying the capture or restoration logic. The BeforeRollback and AfterRollback lifecycle hooks integrate with the existing HookSystem [2], allowing future extensions such as audit logging and custom rollback policies to be developed independently. The RollbackPoint metadata structure includes a version field for forward-compatible schema evolution. The design is also compatible with future features such as partial undo, selective file restoration, and rollback preview.

Testability

The Rollback Manager supports testability at three levels. At the unit level, the SnapshotStore operates on the filesystem through a configurable base path, allowing tests to use temporary directories without affecting the production snapshot location. The RollbackManager itself can be tested with mock instances. At the integration level, the enhancement introduces testable invariants: after a successful /undo, the file content must exactly match the pre-execution snapshot, and after /undo-turn, all files modified during the turn must match their pre-turn state. At the system level, the two new slash commands /undo and /undo-turn can be exercised in end-to-end test suites without modifying the existing test infrastructure.

Performance

The primary performance concern is the overhead of snapshot capture during tool execution. For typical source code files under 100 KB, the overhead is sub-millisecond and imperceptible relative to Gemini API latency. For large files, a configurable threshold defaulting to 10 MB skips snapshot capture to prevent degradation. The in-memory RollbackPoint index adds negligible memory overhead, since file content is stored on disk rather than in memory. Rollback execution has comparable performance to the original tool execution. The enhancement operates exclusively at the tool execution boundary and does not affect other architectural layers.

Architectural Changes

High-Level Architecture Impact

The enhancement introduces a new subsystem, the Rollback Manager, into the high-level architecture established in A2 [2]. The revised architecture shown in Figure 3 also incorporates two components that were absent from our original A2 submission but identified during A2 feedback: Security & Configuration and Authentication / Identity. These components are present in the reference architecture and are included here for completeness, although they are not directly modified by the proposed enhancement. The Rollback Manager itself sits alongside the existing Tool Orchestration and Policy & Confirmation subsystems.

The diagram gains one new component and three new dependency edges. First, Core Runtime depends on Rollback Manager for /undo and /undo-turn command dispatch. Second, Tool Orchestration depends on Rollback Manager for pre-execution snapshot triggers. Third, the Rollback Manager depends on the File System for snapshot storage and file restoration. No existing dependency edges are removed or modified. The CLI Frontend's dependency on Core Runtime for rendering rollback summaries flows through the existing coreEvents interface and does not create a new architectural dependency.

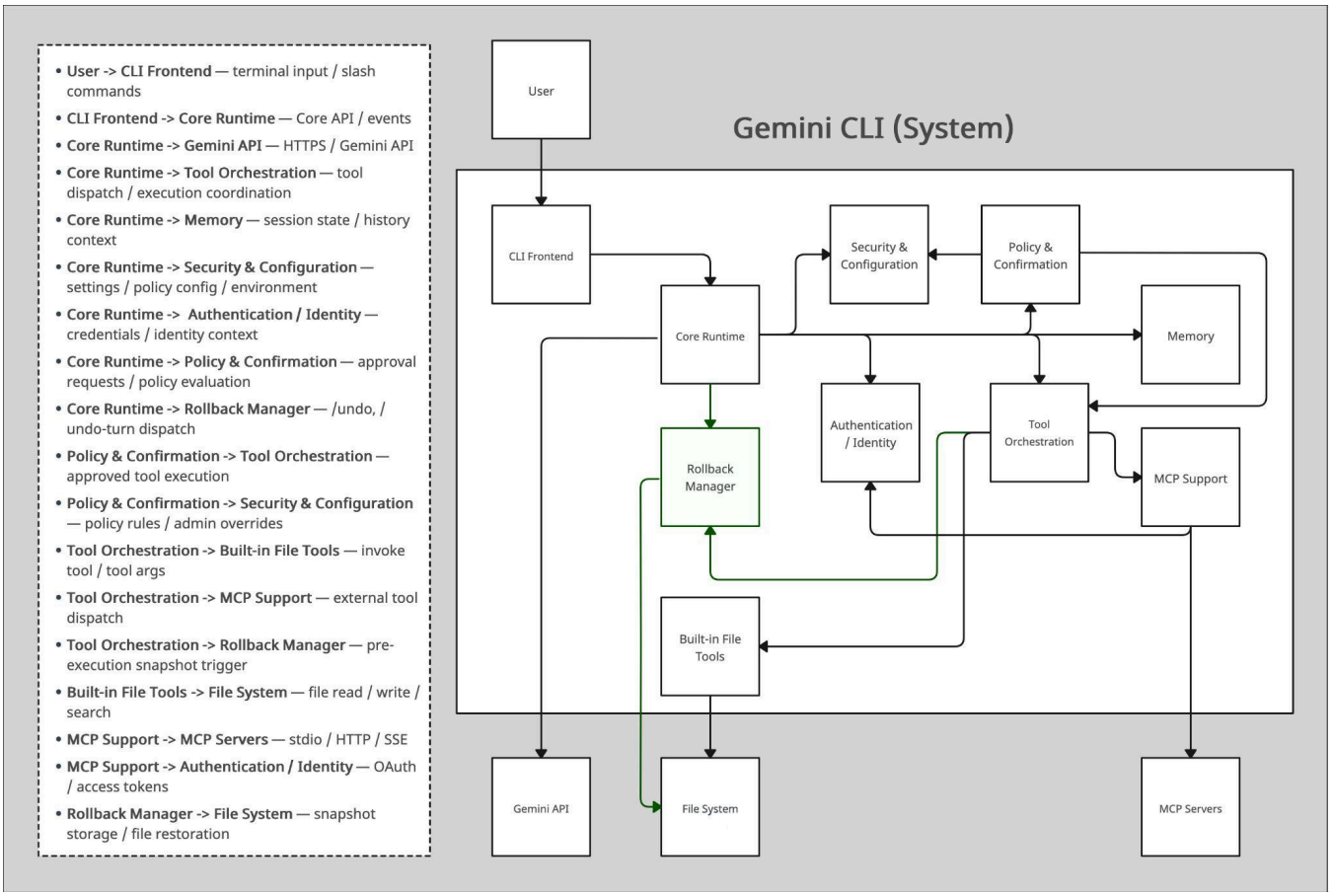

Figure 3: *Updated high-level architecture for Gemini CLI with the proposed Rollback Manager subsystem highlighted in green. The diagram includes the Security & Configuration and Authentication/Identity components added based on A2 feedback. The legend on the left documents all dependency edges and their purposes.*

Low-Level Architecture Impact in packages/core

The Rollback Manager integrates with the six-layer core architecture identified in A2 [2] as follows.

Layer 1: Facade. GeminiClient gains two new public methods, undoLastAction() and undoTurn(turnId). Both delegate to the RollbackManager and return RollbackResult objects. This maintains the Facade pattern [11], so the CLI layer continues to interact only with GeminiClient. Two new event types, RollbackCompleted and TurnRollbackCompleted, are added to coreEvents.

Layer 2: Chat. This layer is not directly modified. The Chat layer handles model interaction and conversation history through GeminiChat, Turn, and ContentGenerator, none of which are affected by the rollback subsystem. After a rollback operation completes, Core Runtime appends a system message to the conversation history via GeminiChat so the model is aware that changes were reverted. This uses the existing message-appending interface and requires no structural changes.

Layer 3: Tool Execution. This is the primary integration point. The snapshot capture hook must run consistently across both the legacy CoreToolScheduler and the event-driven Scheduler. At the transition into the executing state, immediately before delegating to ToolExecutor, the runtime invokes RollbackManager.captureSnapshot(toolCall). The snapshot captures the current state of all resources the tool will modify, identified from the tool's parameter schema such as the path fields for write_file or replace. After successful execution, the snapshot is associated with the tool call ID and turn ID in the RollbackManager's index.

Layer 4: Safety. The PolicyEngine [9] gains a new built-in policy rule. The rollback_operations rule auto-approves rollback actions by default since they restore previously approved states. Enterprise administrators can override this rule to require confirmation. The HookSystem gains two new lifecycle boundaries, BeforeRollback and AfterRollback, allowing hooks to observe or modify rollback behavior for purposes such as enterprise audit logging.

Layer 5: Multi-Agent Framework. Subagent tool executions are captured in the same rollback store and tagged with the agent execution context. The LocalAgentExecutor must propagate the parent's RollbackManager instance to the subagent's CoreToolScheduler so that /undo-turn on the parent turn correctly reverts subagent-induced changes.

Layer 6: Model Routing. This layer is unaffected. The ModelRouterService operates at the prompt-to-model mapping level and has no interaction with tool execution state or filesystem operations. No changes are required.

Implementation Alternatives

Alternative A: Snapshot-Based Rollback

The Snapshot-Based approach captures a byte-level copy of each affected file before any tool modifies it. Snapshots are stored in .gemini/rollback/<session-id>/<turn-id>/ with JSON metadata linking each snapshot to its originating tool call. The RollbackManager maintains an in-memory index mapping turn IDs and tool call IDs to snapshot file paths.

Capture: Before ToolExecutor.execute() runs, the active tool scheduler calls RollbackManager.captureSnapshot(toolCall). The RollbackManager inspects the tool's parameters to identify target files. For each file, it copies the current content to the snapshot directory. For file creation operations, it records a file-did-not-exist marker. For directory operations, it records the directory listing.

Rollback: On /undo, the RollbackManager retrieves the most recent snapshot and restores each file by overwriting with the snapshot copy. Created files are deleted. Restoration bypasses the normal ToolExecutor pipeline and writes directly to the filesystem.

Storage: Snapshots are retained for the session duration and automatically cleaned up when the session ends. The maximum size is configurable with a default of 50 MB. The oldest snapshots are pruned when the limit is reached.

Strengths: Simple, reliable, and guarantees exact restoration. No dependency on individual tools implementing undo logic. Works for all file-based tools including MCP tools without protocol changes.

Weaknesses: Storage overhead is higher for large files. Cannot undo non-file side effects such as network requests or git pushes. Shell commands with side effects beyond file changes cannot be fully reversed.

Alternative B: Operation-Log Based Rollback

The Operation-Log approach requires each tool to register a structured inverse operation alongside its forward execution. The system maintains a transaction log of operations and their computed inverses. Rollback replays inverse operations in reverse chronological order.

Capture: Each tool extends BaseDeclarativeTool with an optional computeInverse(params, result) method. ToolExecutor calls computeInverse() and appends the returned InverseOperation to the transaction log. For example, the replace tool's inverse records the original content of the edited region. The write_file tool's inverse records either a delete operation or the original file content depending on whether the file existed.

Rollback: On /undo, the RollbackManager pops the most recent InverseOperation and executes it through a lightweight pipeline that validates the operation but skips model interaction and approval. For /undo-turn, all inverse operations for the turn are replayed in reverse order.

Strengths: Lower storage overhead. Can handle a broader class of operations including shell command compensation. Supports partial undo. The transaction log provides a natural audit trail.

Weaknesses: Significantly more complex. Every tool must implement computeInverse(), creating an ongoing maintenance burden. MCP tools cannot participate without protocol extensions. Inverse operations may fail if files were externally modified between execution and undo. Shell command inverses are inherently unreliable.

SAAM Analysis

We follow the SAAM methodology defined by Kazman et al. [5], proceeding through stakeholder identification, scenario development as non-functional requirements, individual evaluation, scenario interaction analysis, and overall assessment.

Step 1: Stakeholder Identification

S1: End-User Developers. Primary users who interact with the undo feature through CLI commands. Their priorities are reliability, responsiveness, and ease of use.

S2: Core Runtime Maintainers. Developers maintaining packages/core. Their priorities are low coupling, clear interfaces, and testability.

S3: Tool Developers. Developers of built-in and MCP tools. Their priorities are low integration burden and backward compatibility.

S4: Enterprise Administrators. Policy configurators in managed environments. Their priorities are auditability, compliance, and governance control.

Step 2: Non-Functional Requirements (Scenarios)

ID	NFR	Stakeholder	Scenario
N1	Reliability	S1, S2	A user invokes /undo after an incorrect file edit. The file must be restored to its exact pre-modification state, byte-for-byte, without corruption or partial restoration.
N2	Performance	S1	Snapshot capture during a tool execution cycle must not add perceptible latency, under 100ms per tool call for files under 1 MB.
N3	Maintainability	S2, S3	A new built-in tool is added. The rollback subsystem must support it without requiring the tool to implement any rollback-specific logic.

N4	Extensibility	S3	A third-party MCP server tool modifies local files. The tool must automatically participate in rollback without protocol changes to MCP [10].
N5	Auditability	S4	An enterprise administrator reviews rollback activity. Every rollback must be logged with full provenance: what was undone, when, original tool call ID, and affected files.
N6	Storage Eff.	S1, S4	A developer runs a long session with many tool calls. Rollback data must not exceed 50 MB and must be automatically cleaned on session end.

Step 3: Individual Evaluation

NFR	Weight	Alt. A: Snapshot-Based	Alt. B: Operation-Log
N1	High	Strong. Full file copies guarantee exact restoration regardless of tool implementation quality.	Moderate. Inverse operations may fail if external changes occurred. Shell command inverses are inherently unreliable.
N2	High	Good. Sub-millisecond for typical source files. Large-file threshold prevents degradation.	Good. Computing inverses is lightweight. Slightly faster for large files.
N3	High	Strong. Self-contained subsystem. Zero changes to existing tools.	Weak. Every tool must implement computeInverse(). Ongoing sync burden.
N4	Med	Strong. All file-based tools including MCP automatically participate.	Weak. MCP tools excluded without protocol extension.
N5	Med	Good. Implicit audit trail via snapshot metadata.	Strong. Explicit structured transaction log.
N6	Low	Moderate. Full copies use more space. Mitigated by session-scoped retention.	Strong. Only diffs are stored. Lower requirements.

Step 4: Scenario Interaction Analysis

SAAM requires identifying interactions between scenarios, where satisfying one NFR affects another [5]. We identify three significant interactions.

N1 Reliability and N6 Storage Efficiency. These scenarios are in direct tension. Alternative A achieves strong reliability by storing full file copies, which increases storage consumption. Alternative B achieves storage efficiency by storing only inverse operations, but at the cost of reliability when those operations fail. The interaction favors Alternative A because reliability failures result in corrupted restoration that cannot be recovered, while storage overuse is bounded and manageable through session-scoped cleanup and size limits.

N3 Maintainability and N4 Extensibility. These scenarios reinforce each other. Both favor Alternative A because a self-contained rollback subsystem inherently enables automatic participation of new tools. In Alternative B, the requirement for each tool to implement computeInverse() means that maintainability and extensibility are both negatively affected by distributing rollback logic across tool implementations rather than centralizing it.

N2 Performance and N1 Reliability. A minor tension exists in Alternative A. The large-file threshold that protects performance creates a gap in reliability for files exceeding the threshold. Files above 10 MB are excluded from snapshot capture and cannot be rolled back. However, this affects a small minority of tool operations since most source files are well under 1 MB.

Step 5: Overall Assessment and Recommendation

Alternative A, the Snapshot-Based Rollback approach, is the recommended choice. It scores strongly on the three highest-weighted NFRs of reliability, performance, and maintainability. Its only significant weakness is storage efficiency, which is the lowest-weighted requirement and is adequately mitigated by session-scoped retention and configurable size limits. The scenario interaction analysis confirms that Alternative A's trade-offs are manageable, while Alternative B's fundamental weaknesses in reliability and maintainability cannot be mitigated without undermining the approach's core design. The Snapshot-Based approach also aligns with Gemini CLI's architectural philosophy of centralizing cross-cutting concerns [1, 2], operating independently of individual tool implementations consistent with how PolicyEngine and CoreToolScheduler manage execution governance.

Impacted Directories and Files

The following table maps impacted files to their architectural layers from A2 [2], providing traceability between the proposed enhancement and the concrete architecture.

File/Directory	Arch. Layer	Change
packages/core/src/rollback/	New subsystem	New directory containing RollbackManager, SnapshotStore, RollbackPolicy, and types
packages/core/src/rollback/rollback-manager.ts	New subsystem	Core class with captureSnapshot, undoLast, undoTurn, getHistory methods
packages/core/src/rollback/snapshot-store.ts	New subsystem	Filesystem I/O for snapshot storage in .gemini/rollback/
packages/core/src/rollback/types.ts	New subsystem	Snapshot, RollbackPoint, RollbackResult, RollbackConfig type definitions
packages/core/src/core/client.ts	L1: Facade	Add undoLastAction and undoTurn methods to GeminiClient
packages/core/src/utils/events.ts	L1: Facade	Add RollbackCompleted and TurnRollbackCompleted event types
packages/core/src/core/coreToolScheduler.ts	L3: Execution	Pre-execution snapshot hook at the executing state transition
packages/core/src/scheduler/scheduler.ts	L3: Execution	Same pre-ToolExecutor snapshot hook for the event-driven scheduler
packages/core/src/scheduler/tool-executor.ts	L3: Execution	Pass tool parameter metadata for file identification
packages/core/src/policy/policy-engine.ts	L4: Safety	Add rollback_operations policy rule

packages/core/src/agents/local-executor.ts	L5: Agents	Propagate parent rollback context to subagent
packages/cli/src/ui/commands/	CLI Frontend	Add /undo and /undo-turn slash command handlers
packages/cli/src/ui/rollback-summary.tsx	CLI Frontend	New Ink component for the rollback confirmation UI

Interaction with Existing Features and Testing Plans

Checkpointing

The checkpointing system [7] saves conversation state and, when enabled, a shadow Git snapshot of project files. The RollbackManager must coordinate with checkpointing so that rollback metadata stays consistent with checkpoints and restore workflows. On session restore, the RollbackManager reconstructs its in-memory index from the persisted rollback store under .gemini/rollback/.

Test plan: Execute tool calls that modify files, then checkpoint the session, close it, and reopen it. Verify that /undo correctly restores files from the persisted snapshots. Test edge cases including checkpointing after a partial undo and restoring a checkpoint where snapshot files have been manually deleted.

Multi-Agent Delegation

Subagent tool calls must be captured in the parent's rollback context so /undo-turn correctly reverts subagent-induced changes [2]. The LocalAgentExecutor must inject the parent's RollbackManager into the subagent's execution context.

Test plan: Delegate a file-editing task to the CodebaseInvestigatorAgent and verify that subagent modifications appear in the parent's rollback history. Execute /undo-turn and verify that both parent and subagent changes revert in the correct reverse order.

Sandbox Execution

When sandboxing is enabled, tool execution may run under Docker, Podman, or macOS sandbox-exec. The RollbackManager must capture snapshots of host-mapped paths rather than sandbox-internal paths to preserve persistent state.

Test plan: Enable sandbox mode and execute file-modifying tools. Verify that snapshots are captured for host-mapped paths. Test sandbox-only paths to verify graceful degradation with an appropriate warning.

PolicyEngine and HookSystem

Rollback operations are governed by dedicated policy rules. The HookSystem fires BeforeRollback and AfterRollback events for lifecycle interception, enabling integrations such as enterprise audit logging.

Test plan: Configure a policy that requires rollback confirmation and verify that /undo triggers the approval workflow. Register a BeforeRollback hook that blocks specific file patterns and verify that the hook is respected.

ChatCompressionService

Rollback metadata must be retained when conversation turns are compressed for token management, since users may need to undo actions from summarized turns.

Test plan: Create a long session that triggers compression. Verify that rollback points from compressed turns remain accessible and functional.

Potential Risks

Security. Snapshots contain copies of potentially sensitive source code. The mitigation is to apply restrictive filesystem permissions of 0700 on .gemini/rollback/ at creation time and document this in the security guide.

Performance. Large files could cause snapshot latency. The mitigation is a configurable size threshold defaulting to 10 MB above which capture is skipped, with optional copy-on-write filesystem support where available.

Maintainability. The new integration point in the tool scheduler increases complexity. The mitigation is to implement the RollbackManager as a strictly optional, fail-open component so subsystem failures cannot cascade into critical execution.

Data consistency. External file modifications between snapshot capture and undo can cause silent overwrite. The mitigation is to compare current file content against the post-tool-execution state before restoring, and require explicit confirmation if discrepancies are detected. In general, this addition is beneficial to the effectiveness and confidence of developers since they can safely experiment with automated tool execution. It minimizes the price of improper tool behavior, does not necessitate manual recovery effort, and amplifies confidence in the independent conduct of the apparatus, thus Gemini CLI is more viable to the real-life development processes.

Data Dictionary

Term	Definition
RollbackManager	Core class coordinating snapshot capture, storage, and restoration. Entry point for /undo and /undo-turn operations.
SnapshotStore	Filesystem adapter handling input and output for storing and retrieving file snapshots in .gemini/rollback/.
RollbackPoint	A record linking a pre-execution file snapshot to a specific tool call ID, turn ID, file path, and timestamp.
Snapshot	The captured file content stored before a tool modifies a resource. For file creation operations, the snapshot records a file-did-not-exist marker instead.
RollbackPolicy	PolicyEngine rule governing whether rollback operations require user confirmation. The default is auto-approve.
RollbackResult	Return type from undoLast and undoTurn methods, containing the operation status, list of restored files, and any warnings.

Conclusions

This report proposed a Tool Execution Rollback and Undo System for Gemini CLI. The enhancement addresses gaps in undo ergonomics and scheduler-integrated rollback that exist relative to optional checkpointing, history-based revert, and manual version control. Two implementation alternatives were evaluated using the SEI SAAM methodology [5]. The Snapshot-Based approach was recommended

due to its superior reliability, lower maintenance burden, and automatic compatibility with all tools including MCP-based tools. The enhancement integrates cleanly into the existing six-layer core architecture [2], introducing one new subsystem with well-defined interfaces and preserving the system's centralized orchestration philosophy. Identified risks in security, performance, maintainability, and data consistency are all addressable through the mitigations described.

Lessons Learned

Analyzing and proposing modifications to a system studied across three assignments revealed that deep architectural knowledge is a prerequisite for responsible enhancement design. Having characterized the tool scheduler state machine in A2, we could precisely identify the integration point at the executing state transition without disrupting the existing lifecycle.

We also learned that the SAAM scenario interaction step was more valuable than the individual NFR evaluation. The individual ratings made both alternatives look defensible, but examining how reliability and storage efficiency interacted directly exposed why one alternative was structurally stronger.

Finally, alternative selection depends less on theoretical elegance than on compatibility with the system's established architectural style. The Operation-Log approach is more sophisticated in isolation, but it conflicts with Gemini CLI's pattern of centralizing cross-cutting concerns rather than distributing them across tool implementations.

References

[1] Group 13, “Gemini CLI – Conceptual Architecture,” Assignment 1, CISC 326, February 2026. Available at: https://iramhasan.github.io/group13/#conceptual-architecture

[2] Group 13, “Gemini CLI – Concrete Architecture,” Assignment 2, CISC 326, March 2026. Available at: https://iramhasan.github.io/group13/#concrete-architecture

[3] Gemini CLI Architecture Documentation. https://geminicli.com/docs/architecture/

[4] Gemini CLI GitHub Repository. https://github.com/google-gemini/gemini-cli

[5] R. Kazman et al., “SAAM: A Method for Analyzing the Properties of Software Architectures,” ICSE 1994.

[6] D. Garlan and M. Shaw, “An Introduction to Software Architecture,” 1994.

[7] Gemini CLI Checkpointing Documentation. https://geminicli.com/docs/cli/checkpointing

[8] Gemini CLI Session Management. https://geminicli.com/docs/cli/session-management

[9] Gemini CLI Policy Engine. https://geminicli.com/docs/core/policy-engine

[10] Model Context Protocol Specification. https://modelcontextprotocol.io/

[11] E. Gamma et al., Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

AI Collaboration Report

AI Member Profile and Selection Process

For this assignment, the group used GPT-5.3 (March 2026) as its AI teammate. A3 required architectural reasoning about design alternatives, systematic SAAM evaluation, and precise mapping of changes to the six-layer core architecture recovered in A2. GPT-5.3 was selected for its strength in maintaining coherence across these interrelated concerns, particularly when outputs needed to align with subsystem responsibilities established in prior deliverables. Older GPT versions and general-purpose code assistants were briefly evaluated but proved less effective at sustaining consistency between the SAAM analysis, architectural impact sections, and impacted file mappings. GPT-5.3 offered the best balance of structured reasoning and academic writing quality for a report centered on architectural analysis.

Tasks Assigned to the AI Teammate

The AI assisted with the following tasks. (1) Architectural gap analysis: analyzing the current safety model's limitations to justify the rollback enhancement selection. (2) SAAM structuring: organizing the five-step analysis covering stakeholder identification, scenario development, individual evaluation, scenario interaction, and recommendation. (3) Alternative comparison: articulating trade-offs between the Snapshot-Based and Operation-Log approaches across non-functional requirements. (4) Impacted file mapping: matching file-level changes to architectural layers from A2. (5) Use case refinement: tightening use case descriptions and verifying sequence diagram consistency. (6) Writing support: improving clarity and terminology consistency. In all cases, the AI accelerated drafting while technical decisions and final content remained with the group.

Interaction Protocol and Prompting Strategy

One team member served as primary prompt engineer and shared AI outputs with the group for discussion and validation. Other members proposed questions, checked outputs against the system architecture, and flagged inconsistencies with A1 and A2 findings. Prompting evolved from broad exploration to targeted queries about specific subsystems such as the CoreToolScheduler state machine, PolicyEngine rules, and HookSystem lifecycle boundaries.

One representative prompt was: “Propose and compare two implementation strategies for a rollback feature in Gemini CLI, referencing the CoreToolScheduler, PolicyEngine, and ToolExecutor components from our A2 analysis, and format the comparison in SAAM-compatible structure.” Initial responses were often too abstract. Follow-up prompts demanded concrete architectural references, explicit layer mappings, and alignment with the recovered concrete architecture. This iterative cycle of broad prompt, evaluate, narrow, and re-prompt produced increasingly accurate outputs.

Validation and Quality Control Procedures

All AI-assisted content was validated before inclusion. Architectural claims were checked against the Gemini CLI repository, official documentation, and our A1 and A2 findings. File paths in the impacted files table were manually verified to confirm existence and correct architectural role. The SAAM analysis was reviewed for logical consistency, with NFR-to-stakeholder mappings, evaluation ratings, and scenario interactions each independently checked by at least two group members. Use cases, sequence diagrams, architectural changes, and testing plans were cross-referenced to ensure mutual consistency. For example, the group verified that every component appearing in a sequence diagram was also listed in the impacted files table. No AI-generated claim entered the final report without traceable verification against a primary source.

Quantitative Contribution to Final Deliverable

The AI contributed approximately 35 to 40 percent of the final deliverable. This was concentrated in early-phase exploration (approximately 15 percent), SAAM structuring and alternative comparison (approximately 12 percent), and writing refinement (approximately 8 to 13 percent). The remaining 60 to 65 percent was human work, including architectural reasoning, source verification, diagram creation, cross-referencing against A1 and A2, and integration of all report elements.

Reflection on Human-AI Team Dynamics

The AI was most valuable during brainstorming and structural organization. It rapidly generated candidate architectural ideas and arranged them into formal frameworks like SAAM. It was particularly effective at articulating trade-offs between design alternatives in a structured format. The primary challenge was that initial outputs were often generic and misaligned with the specific architecture of Gemini CLI, requiring multiple refinement iterations. There was also a recurring risk of accepting plausible-sounding outputs that were not fully accurate, which reinforced the importance of our validation procedures. The most effective workflow used AI for idea generation and structural scaffolding, with the group handling verification, refinement, and final decisions. The key lesson is that AI works best as a guided drafting partner rather than an authoritative source, and its ability to accelerate exploration must be paired with rigorous human verification against the actual system under analysis.